Homework Submission

> Updating the Playwright Test-Generator Agent to Produce Framework-Conformant Tests — Design, Implementation & Verification

---

Project: `/Users/mohit.kumar/LearningMaterials/PlayWrightAgents`
Reference framework studied: SonicFramework (Page-Object / fixture / wrapper conventions)
Application under test: TodoMVC (`demo.playwright.dev/todomvc`)

Contents

1. Assignment

Task given: "Update the Playwright Generator Agent prompt to generate test scripts according to the given framework format." A reference framework ("SonicFramework") was provided, containing a Page Object Model layer, a shared browser-action wrapper class, custom Playwright fixtures, a centralized locators file, and framework-specific naming/annotation conventions built originally for a CRM demo application (Leaftaps).

The generator agent in question — `.claude/agents/playwright-test-generator.md` — is one of three cooperating agents in this project (planner → generator → healer). Before this change, it produced fully self-contained, standalone Playwright tests: raw `page.*` calls, plain `test.describe` / `test` blocks, no Page Objects, no fixtures.

2. Approach

1. Studied the reference framework's **actual code** (not its own stale README, which described a much simpler, unused pattern) to extract the real, enforced conventions: Page Object Model, a shared `PlaywrightWrapper` action layer, `test.extend()` fixtures, a nested-section locators file, data-driven JSON test data, and a `TC00N` naming/ annotation convention.
2. Rewrote the generator agent's prompt to encode these conventions as explicit, ordered instructions (Section 3).
3. Implemented the same building blocks directly inside this project (rather than in a separate external repository) so that the agent, its MCP tooling, and the generated tests all live in one self-contained, correctly-scoped project (Section 4).
4. Used the updated agent to actually generate a full 24-scenario test suite for TodoMVC — not just reviewed the prompt on paper, but proved it end-to-end by running the generated tests for real (Sections 6, 9).
5. Diagnosed and fixed every real failure encountered along the way, including one genuinely flaky test resolved by invoking the project's own `playwright-test-healer` agent (Sections 7, 8).

3. Change 1 — Updating the Generator Agent's Prompt

File changed: `.claude/agents/playwright-test-generator.md`

Before

```
# For each test you generate
- Obtain the test plan with all the steps and verification specification
- Run the `generator_setup_page` tool to set up page for the scenario
- For each step and verification in the scenario, do the following:
  - Use Playwright tool to manually execute it in real-time.
  - Use the step description as the intent for each Playwright tool call.
- Retrieve generator log via `generator_read_log`
- Immediately after reading the test log, invoke `generator_write_test`
  with the generated source code
  - File should contain single test
  - Test must be placed in a describe matching the top-level test plan item
  - ...

Example output:
test.describe('Adding New Todos', () => {
  test('Add Valid Todo', async ({ page }) => {
    await page.click(...);
  });
});
```

After (key additions)

The rewritten prompt keeps the exact same live-execution mechanism (setup page → execute steps live in the browser → read the execution log → write the test) — that part was never broken and was deliberately left untouched. What changed is what happens *between* reading the log and writing the file:

1. **A framework-inspection step**, run before touching the browser: read the shared wrapper class's method vocabulary, check whether a Page Object/fixture already exists for the app under test, learn the locators-file's nested-section convention, and find the next free `TC00N` number.
2. **A "codify into framework shape" step**, run after execution but before writing the test: reuse existing selectors/methods where they already cover the interaction;

otherwise add a new selector, a new Page Object method (built only from the shared wrapper's methods, never raw `page.*`), or — for a brand-new app — a whole new Page Object class + fixture file.

3. **Precise file-writing rules**: import `test` only from the app's fixture file (never `@playwright/test`), `test.info().annotations.push(...)` as the first statement, `TC00N_<name>.spec.ts` naming, and a fallback to a direct `Write` if the MCP tool's own write path is rejected.
4. **An explicit "Must not" list**: no raw `page.*` calls in specs, no `new SomePage(page)` inside a test, no locator strings outside the locators file, never force-fit one app's fixtures onto an unrelated app.
5. **Frontmatter change**: added `Write, Edit` to the agent's allowed tools list, since it must now create/modify Page Object, fixture, and locator files — not only test files.

The current, full prompt is reproduced in the appendix file `.claude/agents/playwright-test-generator.md` for direct grading reference.

4. Change 2 — Framework Building Blocks Implemented

The following framework layer was built inside this project, directly mirroring the reference framework's structure:

Building block	File	Role
Shared action/wrapper layer	`helpers/playwright.ts`	Class `PlaywrightWrapper` — the only place that touches the raw Playwright `page` API; every Page Object extends it
Page Object Model	`pages/todoMvcAppPage.ts`	Class `TodoMvcAppPage extends PlaywrightWrapper` — one method per user action/verification (~30 methods, covering add/edit/delete/toggle/filter/persist)
Centralized locators	`pages/selectors.ts`	Nested `todoMvc: { ... }` section holding every selector used by the page object
Fixtures	`customFixtures/todoMvcFixture.ts`	`test.extend()` — hands each test a ready `TodoMvcApp` object
Generated tests	`tests/TC001…TC024*.spec.ts`	24 framework-conformant test files, one scenario each

Fixture file (in full, for reference)

```
import { test as baseTest } from '@playwright/test'
import { TodoMvcAppPage } from '../pages/todoMvcAppPage'

type todoMvcFixture = {
    TodoMvcApp: TodoMvcAppPage
}

export const test = baseTest.extend<todoMvcFixture>({
    TodoMvcApp: async ({ page, context }, use) => {
        const todoMvcApp = new TodoMvcAppPage(page, context);
```

```
        await use(todoMvcApp);
    },
})
```

Page Object excerpt

```
export class TodoMvcAppPage extends PlaywrightWrapper {
    constructor(page: Page, context: BrowserContext) {
        super(page, context);
    }
    public async addTodo(value: string) {
        await this.typeAndEnter(selectors.todoMvc.newTodoInput, "New Todo", va
    }
    public async verifyItemCounter(expectedValue: string) {
        await this.verification(selectors.todoMvc.todoCount, expectedValue);
    }
    // ...~28 more methods: toggle, delete, edit, filter, persistence, etc.
}
```

> **Design decision — where the framework lives:** the framework layer was implemented directly inside this project (rather than in the separately provided reference-framework repository), so that the generator agent, its MCP browser-automation tooling, and the tests it produces all operate within one correctly-scoped project. This avoided a real, encountered problem: when a first attempt wrote generated tests into an external sibling project, the MCP tool's own file-write validation was scoped to the wrong project root, producing a rejected write and requiring a manual fallback. Keeping everything in one project removes that class of problem entirely.

5. Before / After: Test Code Style

This project retains the original, pre-assignment standalone tests alongside the new ones, giving a direct side-by-side comparison of the same scenario written both ways.

Before — standalone style (`tests/adding-todos/add-a-single-todo-via-enter-key.spec.ts`)

```
import { test, expect } from '@playwright/test';

test.describe('Adding Todos', () => {
  test('Add a single todo via Enter key', async ({ page }) => {
    await page.goto('https://demo.playwright.dev/todomvc/');
    const newTodoInput = page.getByRole('textbox', { name: 'What needs to be d
    await newTodoInput.click();
    await newTodoInput.fill('Buy milk');
    await newTodoInput.press('Enter');
    await expect(page.getByTestId('todo-title')).toHaveText('Buy milk');
    await expect(page.getByTestId('todo-count')).toContainText('1 item left');
  });
});
```

After — framework-conformant style (`tests/TC001_add_todo_via_enter_key.spec.ts`)

```
import { test } from "../customFixtures/todoMvcFixture"

test.describe('Adding Todos', () => {
    test('Add a single todo via Enter key', async ({ TodoMvcApp }) => {
        test.info().annotations.push({ type: 'TestCase', description: 'Add a s

        await TodoMvcApp.loadApplication('https://demo.playwright.dev/todomvc/
        await TodoMvcApp.verifyEmptyTodoList()
        await TodoMvcApp.verifyNewTodoPlaceholder('What needs to be done?')
        await TodoMvcApp.verifyFooterHidden()

        await TodoMvcApp.addTodo('Buy milk')
        await TodoMvcApp.verifyTodoAdded('Buy milk')
        await TodoMvcApp.verifyTodoUnchecked()
        await TodoMvcApp.verifyItemCounter('1 item left')
        await TodoMvcApp.verifyFooterVisible()
        await TodoMvcApp.verifyNewTodoInputCleared()
    })
})
```

No raw `page.*` calls, no inline locator strings, framework-mandated annotation line present, imports the fixture-scoped `test` rather than `@playwright/test`.

6. Generated Test Suite (24 Scenarios)

All 24 scenarios from the pre-existing test plan (`specs/todomvc.plan.md`) were regenerated through the updated agent, across 6 suites:

Suite	Scenarios
Adding Todos	Add via Enter, multiple todos + pluralization, empty/whitespace rejected, text trimming (5)
Editing Todos	Save via Enter, cancel via Escape, delete-on-empty, edit while completed, blur-to-save (5)
Completing and Toggling Todos	Single toggle, toggle-all, mixed-state toggle-all (3)
Deleting Todos and Clear Completed	Delete via hover-x, delete-last-item empty state, clear-completed present/hidden (4)
Filters and Routing	All/Active/Completed subsets, deep-linking, empty completed-filter view, add-while-filtered (4)
Persistence and Reload Behavior	List persists, filter persists, empty state persists (3)

7. Issues Found & Fixed During Verification

The prompt rewrite was not just reviewed on paper — the agent was actually run, and every resulting failure was root-caused and fixed rather than worked around. Three issues were diagnosed and fixed directly; a fourth (Section 8) was fixed by invoking the project's own healer agent.

Issue 1 — Shared `fetchattribute()` helper returned `undefined`

> **Root cause:** the helper called `elementHandle.evaluate(node => node.getAttribute(attName))`. Playwright serializes this callback to run inside the browser context, so it cannot close over the outer `attName` variable from Node.js — it must be passed explicitly as an `evaluate` argument.
> **Fix (in the one shared method, benefiting every caller):**

```
// before
return eleValue?.evaluate(node => node.getAttribute(attName))
// after
return eleValue?.evaluate((node, attName) => node.getAttribute(attName),
```

Issue 2 — Shared `doubleClick()` helper didn't trigger real double-click behavior

> **Root cause:** the method fired two separate `.click({force:true})` calls, which does not reliably register as a native browser `dblclick` event — confirmed live: the target element never gained its "editing" state.
> **Fix:** replaced with a real `.dblclick({force:true})` call, flagged with a `ponytail:` comment noting the ceiling and that zero existing callers relied on the old behavior (verified via search before changing shared code).

Issue 3 — One generated test asserted behavior the application doesn't actually have

> **Root cause:** a test asserted the new-todo input field is cleared after submitting a whitespace-only (rejected) entry. Live behavior shows the app only clears the field on a *successful* add, not a rejected one — the assertion was simply wrong about real app behavior.

> **Fix:** removed the incorrect assertion from that one test file only (not a shared method, so no other test was affected); the scenario's real intent — "no todo gets persisted" — was already covered by the two prior assertions in the same test.

8. How the Healer Agent Fixed the Flaky Test (Detailed)

One test — verifying that the All/Active/Completed filters show the correct subset of todos — failed intermittently. Rather than diagnosing it manually, the project's own `playwright-test-healer` agent was invoked against the live suite, exactly as it would be used in normal day-to-day maintenance.

What the healer was asked to do

Run the full suite, treat the flaky test as a real defect to root-cause (not paper over with a sleep), and — since the underlying assertion helper is shared by many tests — fix the shared method rather than patching the one symptomatic test file.

The healer's diagnosis

Root cause identified: the shared `verification()` helper in `helpers/playwright.ts` read an element's `innerText()` *exactly once* and compared it manually with a plain `expect(text).toContain(...)` — a one-shot check with no retry. The page object's `verifyTodoAtPosition()` method (used by the filter test) calls this shared helper. Clicking a filter link only waits for the link itself to be clickable — it does not, and structurally cannot, know how long the single-page application takes to finish re-rendering the filtered list afterward. Because `verification()` never retried, it sometimes captured the DOM *mid-re-render* — still showing the pre-filter list — producing exactly the observed symptom: expecting "Walk the dog" at position 1 after clicking "Active," but reading back "Buy milk" (the stale, unfiltered first item).

The healer's fix

Fixed in the single shared method — not in the test, not in the click methods, not by adding a blind `waitForTimeout()` :

```
// before — one-shot read, no retry
async verification(locator: string, expectedTextSubstring: string) {
    const element = this.page.locator(locator).nth(0);
```

```
    const text = await element.innerText();
    expect(text.toLowerCase()).toContain(expectedTextSubstring.toLowerCas
}

// after — Playwright's built-in auto-retrying assertion
async verification(locator: string, expectedTextSubstring: string) {
    const element = this.page.locator(locator).nth(0);
    await expect(element).toContainText(expectedTextSubstring, { ignoreCa
}
```

`expect(locator).toContainText()` polls the element's text for up to the configured timeout instead of reading it once, so it now correctly waits out the application's post-filter re-render instead of racing it.

Why this fix, and not an alternative

- **No blind sleep/timeout was added** — that would have masked the race instead of resolving it, and would have slowed down every test using the method regardless of whether it needed the wait.
- **The click methods (`clickActiveFilter`, etc.) were correctly left unchanged** — they aren't the root cause; requiring every caller to know how long a re-render takes would leak a UI-timing concern into unrelated code.
- **Because the fix landed in the one shared method**, every test that uses `verification()` (dozens of call sites: item counters, todo titles, filter labels, etc.) benefits at once — this is precisely the "fix the shared function once" principle a proper framework is designed to make possible.

Verification of the fix

The healer re-ran the full suite twice after the fix (all passing both times), then specifically re-ran the four filter-related tests together, then the originally-flaky test alone a further time — confirming the flakiness was genuinely resolved, not incidentally avoided by test ordering.

9. Final Verification Results

Independent final run, after all fixes above, executed directly (not just trusting agent self-reports):

```
$ npx playwright test

49 passed (1.4m)
```

49 / 49 PASSED — the original 24 standalone tests, the 24 new framework-conformant tests (`TC001–TC024`), and the project's seed test, all green in a single run. An HTML report was additionally generated (`npx playwright test --reporter=html`) and visually reviewed.

10. Known Gaps & Recommendations

In the interest of a complete, honest submission, the following operational pieces were intentionally out of scope for this assignment and remain open:

Gap	Recommendation
No `playwright.config.ts`	Add one to control retries, timeouts, base URL, and default reporter, instead of relying on tool defaults
No CI pipeline	Wire `npx playwright test` into a workflow that runs on every relevant code change
No environment/URL constants file	Centralize the TodoMVC base URL instead of repeating the literal string per test
No data-driven layer in use	Not required by any of the 24 current scenarios; the convention is documented in the agent prompt and ready to use once a data-driven scenario appears

---

Submitted as evidence of the completed assignment: the agent's updated prompt, the framework it now targets, the 24 tests it generated, and the real defects found and fixed (directly, and via the project's own healer agent) while proving the whole pipeline actually works end-to-end rather than only reading correctly on paper.